

ARQUITECTURA Y METODOLOGÍA

Waggo por dentro

Cómo está construido el producto y cómo se trabaja con inteligencia artificial y desarrollo guiado por especificaciones (SDD).

Cifras contadas del repositorio el 28 de octubre de 2026 · cada lámina dice cómo.

EL PRODUCTO

Un producto, tres repositorios y un vault

Waggo

Waggo conecta dueños de perros con paseadores verificados: el dueño pide el paseo, lo sigue en vivo por GPS y paga cuando termina bien.

- **waggo-api** · .NET 10, Clean Architecture, PostgreSQL + PostGIS
- **waggo-mobile** · Expo SDK 57 · iOS, Android y web
- **waggo-workspace** · specs, agentes, skills y reglas (sin código de producto)
- **Vault Waggo** · Obsidian: requisitos, arquitectura y ADR

44
endpoints en 9 módulos

14
features en la app móvil

21
specs, 18 ya implementadas

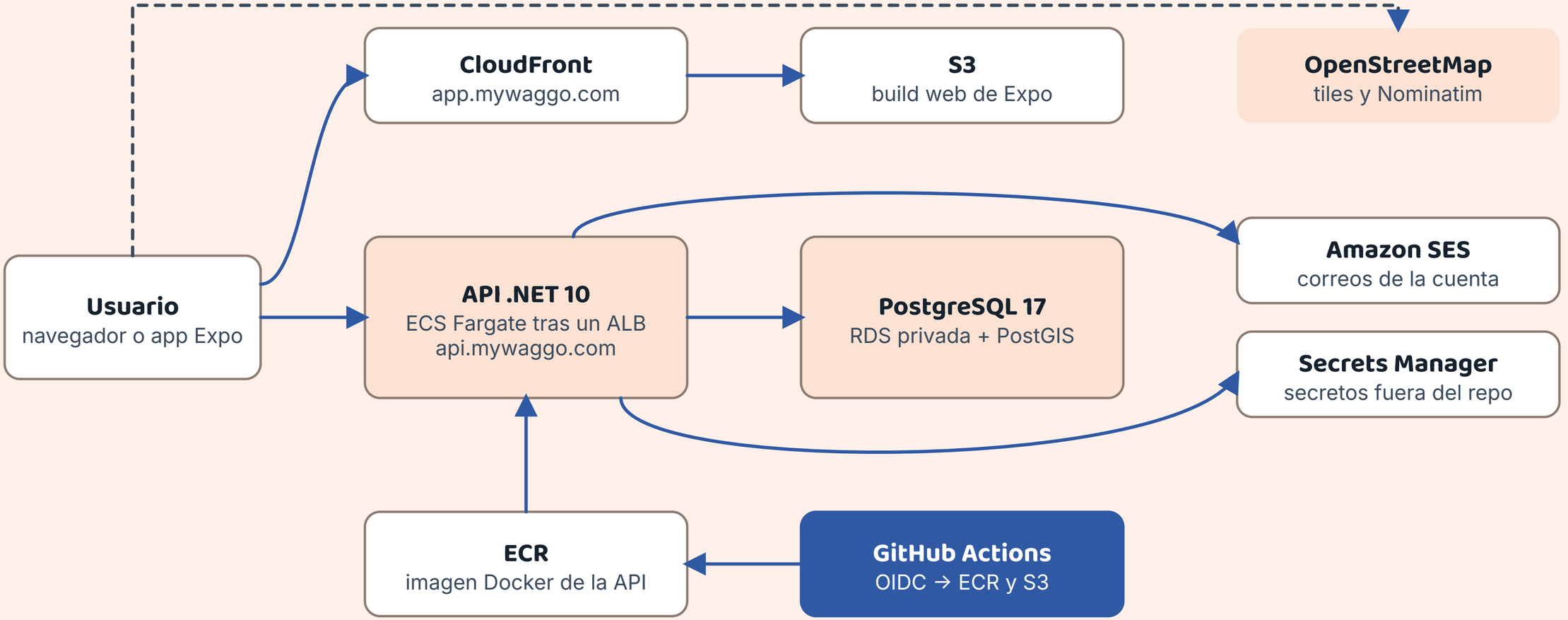
19
migraciones de base de datos

Cómo se contó: endpoints = llamadas MapGet/MapPost/MapPut en waggo-api/Waggo.Api; módulos = carpetas de Waggo.Api/Endpoints; features = carpetas de waggo-mobile/src/features; specs = archivos de specs/ sin plantilla, README ni borrador de ADR; migraciones = archivos de Migrations sin Designer ni Snapshot.

ARQUITECTURA

Cómo se despliega hoy en AWS

Waggo

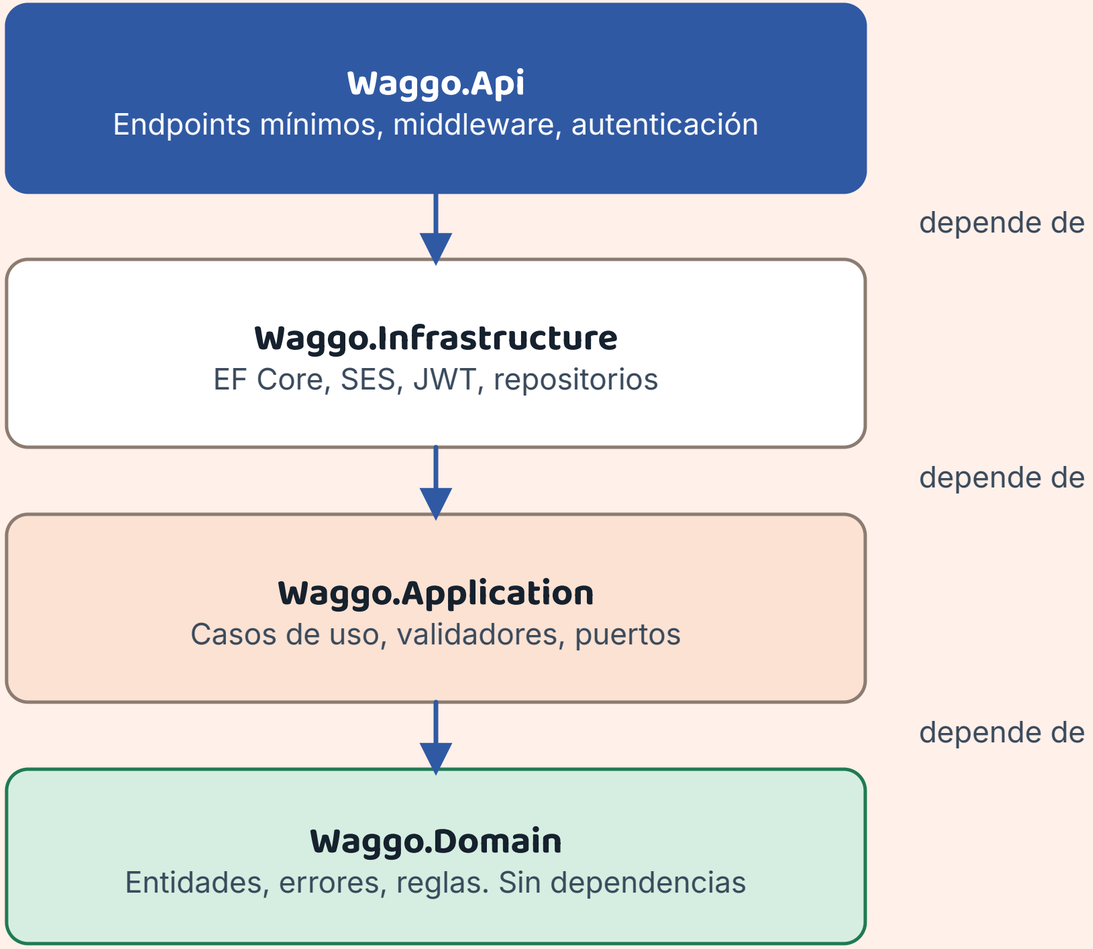


Fuente: specs/INF-001-despliegue-publico-en-aws.md (estado Implementada, ADR-020). Comprobado el 2026-10-28: https://app.mywaggo.com responde 200 y https://api.mywaggo.com/health/ready responde 200.

WAGGO-API

Clean Architecture con la regla verificada por pruebas

Waggo



La regla **Api → Infrastructure → Application → Domain** no es solo una convención: la comprueban pruebas de arquitectura (NetArchTest) que fallan si se rompe.

- **4 proyectos** de producción y **6** de pruebas
- Cada caso de uso es un **Command o Query + Handler + Validator**, sin MediatR ni AutoMapper
- Todo devuelve **WaggoResponse<T>** con errores, avisos e información
- Carpetas por tipo y por módulo (ADR-012): 9 módulos de endpoints

Fuente: waggo-api/CLAUDE.md, waggo-api/Tests/Waggo.ArchitectureTests/LayerTests.cs, Directory.Packages.props.
Proyectos = carpetas Waggo.* (4) y Tests/* (6).

Una emergencia, de la petición a la respuesta

Waggo



RaiseEmergencyHandler.cs (fragmento)

```
if (walk.Status is not (WalkStatus.Accepted
    or WalkStatus.InProgress))
{
    response.AddError(AlertErrors.WalkNotActive);
    return response;
}
```

AlertErrors.cs (catálogo de errores del dominio)

```
public static readonly Error WalkNotActive = new(
    "Alerts.WalkNotActive",
    "Solo puedes reportar una emergencia en un
     paseo aceptado o en curso.",
    ErrorType.BusinessRule);
```

Un error de negocio esperado se **acumula** en la respuesta (código `Alerts.WalkNotActive`, HTTP 422); si la operación no puede seguir, se lanza una excepción propia que el middleware traduce, sin exponer el detalle técnico.

Fuente: waggo-api/Waggo.Application/Tracking/Commands/RaiseEmergency/RaiseEmergencyHandler.cs y Waggo.Domain/Errors/Tracking/AlertErrors.cs (fragmentos reales); waggo-api/CLAUDE.md (ADR-007, ADR-009, ADR-010).

DATOS

PostgreSQL + PostGIS y mapas abiertos

Waggo

PostgreSQL + PostGIS

- EF Core con **snake_case** y un esquema por módulo
- **19 migraciones**; se aplican al iniciar solo en Development y Testing
- Matching por cercanía en la base: solicitudes a menos de **5 km**, orden por distancia real y índice GiST (RF-006)
- Documento del paseador y notas médicas **cifrados** en la base
- Pruebas de integración contra **PostgreSQL real** (Testcontainers), nunca SQLite ni InMemory

Mapas con OpenStreetMap

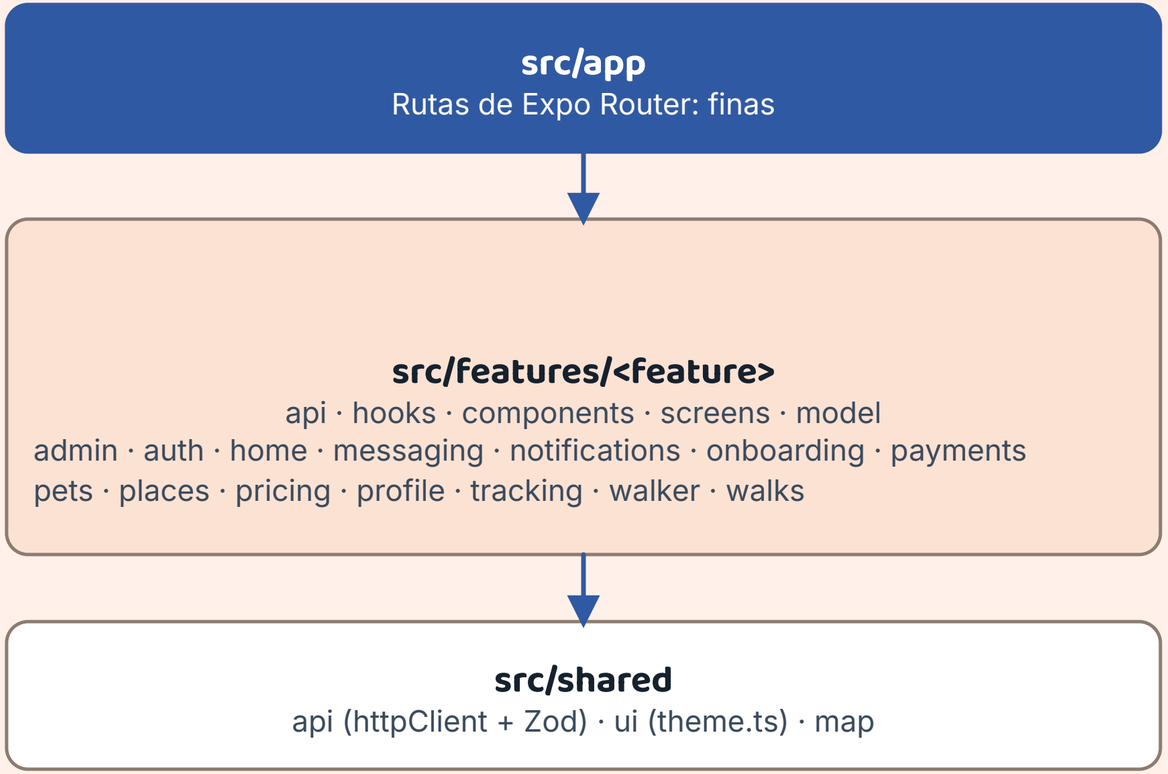
- Decisión del PO (2026-10-27): **OpenStreetMap**, sin claves ni costo de licencia
- Un componente **WaggoMap**: Leaflet dentro de WebView (móvil) o iframe (web)
- Mensajes app ↔ mapa **tipados y validados con Zod**
- Búsqueda de direcciones con **Nominatim**: máx. 1 petición por segundo y solo al enviar
- Atribución «© OpenStreetMap contributors» siempre visible

Fuente: waggo-api/CLAUDE.md, specs/RF-006-matching-por-cercania.md, specs/_adr-borrador-mapas-openstreetmap.md (borrador de ADR: aún no pasó al vault), .claude/rules/mobile-checklist.md. Los tiles públicos de OSM son solo para desarrollo y piloto.

WAGGO-MOBILE

Expo y React Native, organizada por features

Waggo



- **Expo SDK 57**, React Native 0.86, React 19 y TypeScript
- Una feature solo se importa por su **index.ts**; ESLint bloquea los atajos
- Llamadas a la API por **httpClient + esquema Zod**; los errores llegan como ApiError
- **TanStack Query** para el estado del servidor
- Solo tokens de **theme.ts**: Seashell & Sapphire, Baloo 2 e Inter (ADR-018)
- 14 features, una por dominio

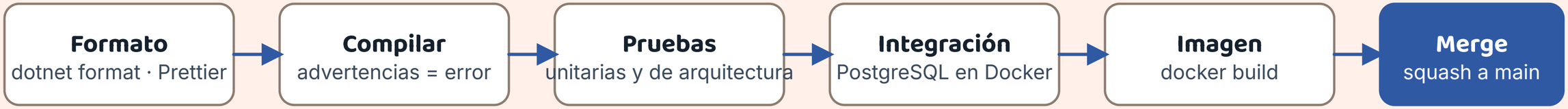
Fuente: waggo-mobile/package.json (versiones), waggo-mobile/AGENTS.md y .claude/rules/mobile-checklist.md; features = carpetas de waggo-mobile/src/features.

ENTREGA CONTINUA

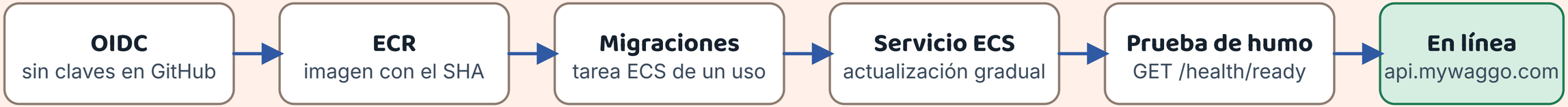
De un pull request a producción, sin pasos manuales

Waggo

1 · En el pull request (CI)



2 · Al llegar a main (deploy de la API)



Web: expo export → S3 → invalidación de CloudFront. Reversa de la API: volver a desplegar un SHA anterior (workflow manual).

Fuente: waggo-api/.github/workflows/ci.yml y deploy.yml, waggo-mobile/.github/workflows/ci.yml, specs/INF-001. Los workflows de CI existen en los dos repos; el título del PR se valida con pr-title.yml (Conventional Commits).

DECISIONES

Las decisiones quedan escritas como ADR

Waggo

ADR	DECISIÓN	DÓNDE SE APLICA
ADR-001	Repositorios de producto separados y un workspace de coordinación	README del workspace
ADR-006	Sin MediatR, AutoMapper ni FluentAssertions: handlers propios y Shouldly	waggo-api
ADR-007	Toda operación devuelve WaggoResponse<T>; el endpoint escribe los logs	waggo-api
ADR-008	Convenciones de código, impuestas por la compilación	.editorconfig
ADR-009/010	Validación en dos capas; errores acumulables o excepciones propias	waggo-api
ADR-011	OAuth 2.0 con JWT y roles owner, walker y admin	waggo-api
ADR-012	Dónde va cada archivo (carpetas por tipo y por módulo)	backend-checklist
ADR-016/017	Identidad propia con bcrypt y JWT; login sin 2FA obligatorio y federación OAuth	specs de identidad
ADR-018/019	Seashell & Sapphire (reemplaza ADR-013); pestañas del paseador	waggo-mobile
ADR-020	Despliegue público en AWS	INF-001

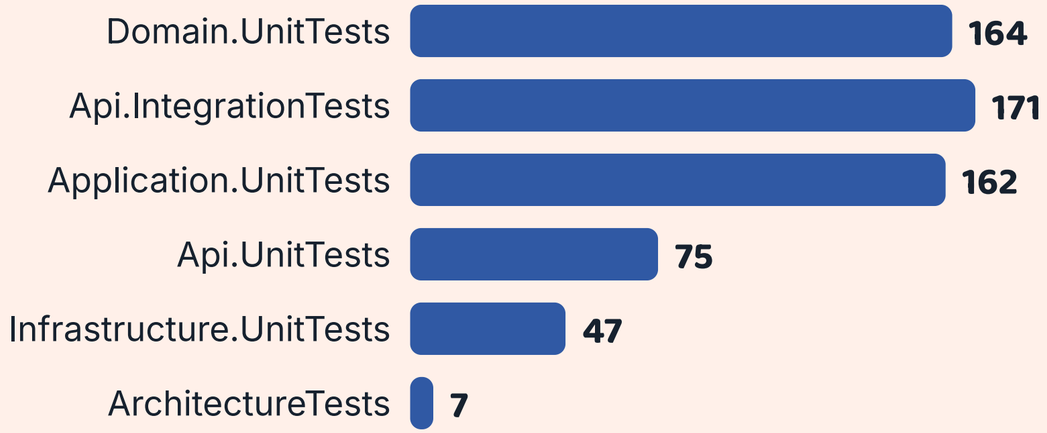
Cómo se contó: los repos citan 18 identificadores ADR distintos (ADR-001, 002, 005 a 020). El vault de Obsidian, donde están los textos completos, no estaba disponible al preparar esta presentación: los títulos son los que aparecen en CLAUDE.md, AGENTS.md y las specs; no se verificó cuántos ADR existen en el vault.

CALIDAD

Casi tanto código de pruebas como de producción

Waggo

waggo-api · 626 métodos de prueba (553 [Fact] + 73 [Theory], 183 casos [InlineData])



466

pruebas en la app (Jest + RNTL) en 95 archivos

114

archivos de prueba en la API

Líneas de código (producción vs pruebas)



Cómo se contó: API = líneas [Fact]/[Theory] bajo Tests/ (grep); App = llamadas it()/test() en *.test.ts(x) (95 archivos; 10 usan .each, que ejecutan varios casos); líneas = suma de .cs sin Migrations ni bin/obj, y .ts/.tsx de src. No se ejecutó la suite: son métodos de prueba, no ejecuciones.

CALIDAD

Las reglas no se piden: la compilación las exige

Waggo

Directory.Build.props (fragmento real)

```
<TreatWarningsAsErrors>true</TreatWarningsAsErrors>
<AnalysisLevel>latest-recommended</AnalysisLevel>
<EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
<Nullable>enable</Nullable>
```

.editorconfig (fragmento real)

```
# var: nunca
csharp_style_var_elsewhere = false:warning
csharp_prefer_braces = true:warning
dotnet_style_namespace_match_folder = true:warning
```

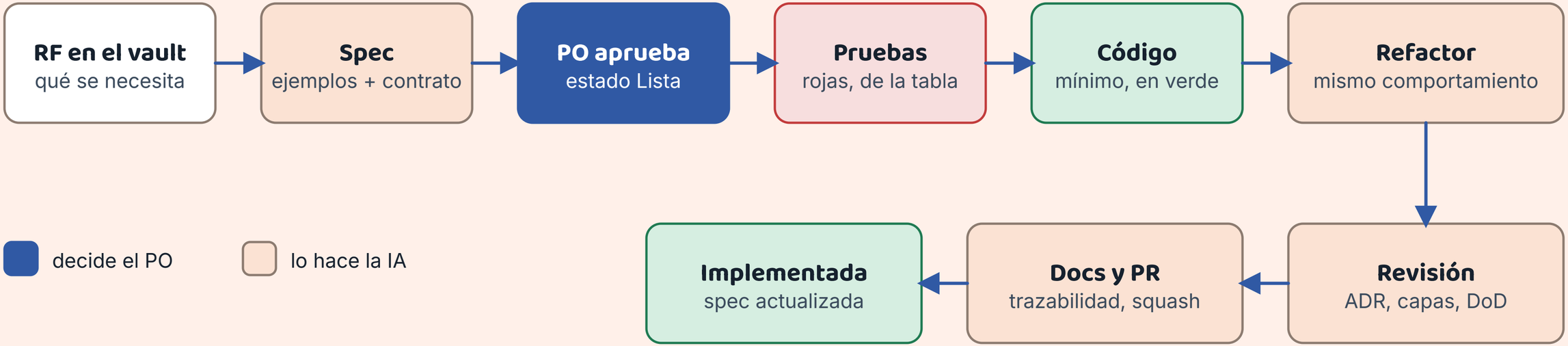
- **API:** sin `var`, llaves siempre, namespace = carpeta, un tipo por archivo; si falla, no compila
- **CI:** `dotnet format --verify-no-changes` y paquetes con vulnerabilidades rompen el build
- **Arquitectura:** pruebas NetArchTest vigilan las capas
- **App:** ESLint (sin `any`, solo exportaciones con nombre, imports de features por index.ts), Prettier y `tsc --noEmit`
- **Commits:** título del PR en Conventional Commits, validado por un workflow

Fuente: waggo-api/Directory.Build.props y .editorconfig (fragmentos reales), .github/workflows/ci.yml y pr-title.yml, waggo-mobile/AGENTS.md, .claude/rules/*.md.

METODOLOGÍA

Spec → pruebas → código → revisión

Waggo



Sin spec en estado Lista no hay código de producto para ese requerimiento.

Fuente: CLAUDE.md del workspace («How work flows here»), specs/README.md (ciclo de vida), .claude/skills/implement-spec/SKILL.md.

METODOLOGÍA

Una fila de la spec, trazada hasta el código

Waggo

① specs/RF-012 · tabla de ejemplos (fila 5)

```
5 | Levantarla en un paseo Requested,
  Completed o Cancelled
  → 422 Alerts.WalkNotActive | Regla
```

② prueba de integración (AlertEndpointTests.cs)

```
[Fact]
public async Task Emergency_OnAWalkNobodyAcceptedYet_Returns422()
{
    ...
    raised.StatusCode.ShouldBe(HttpStatusCode.UnprocessableEntity);
    await WalkScenario.ShouldHaveSingleErrorAsync(
        raised, "Alerts.WalkNotActive");
}
```

③ catálogo de errores (AlertErrors.cs, dominio)

```
WalkNotActive = new("Alerts.WalkNotActive",
    "Solo puedes reportar una emergencia en un
     paseo aceptado o en curso.", ErrorType.BusinessRule);
```

④ código de producción (RaiseEmergencyHandler.cs)

```
if (walk.Status is not (WalkStatus.Accepted
    or WalkStatus.InProgress))
{ response.AddError(AlertErrors.WalkNotActive); return response; }
```

Cada fila de la tabla es al menos una prueba; el nombre del error es el hilo que une spec, prueba y código. La spec cierra con una tabla de **trazabilidad** de archivos y pruebas por repo.

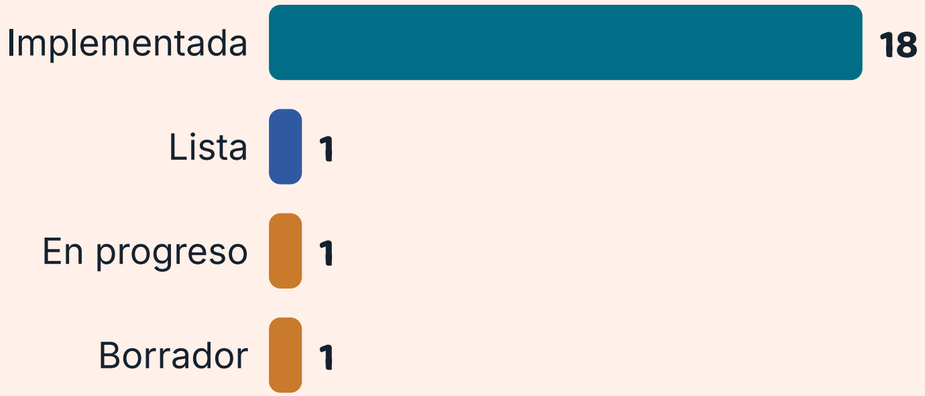
Fuente: specs/RF-012-alerta-de-emergencia.md; waggo-api/Tests/Waggo.Api.IntegrationTests/Endpoints/Tracking/AlertEndpointTests.cs (también AlertHandlersTests.cs en Application.UnitTests); Waggo.Domain/Errors/Tracking/AlertErrors.cs; Waggo.Application/Tracking/Commands/RaiseEmergency/RaiseEmergencyHandler.cs. Fragmentos reales, abreviados.

METODOLOGÍA

21 specs, 18 ya implementadas

Waggo

Estado de las specs



Ciclo de vida: **Borrador** → **Lista** (la aprueba el PO) → **En progreso** → **Implementada**

Qué lleva cada spec

- Historia de usuario y criterios de aceptación
- **Tabla de ejemplos**: feliz, error, borde, seguridad
- Reglas de negocio; las sin confirmar se marcan ⚠️ **provisional**
- Contrato de API y diseño por capas
- Plan TDD por repo y **preguntas abiertas** para el PO
- Trazabilidad: archivos y pruebas reales

Cómo se contó: archivos de specs/*.md sin _template, README ni _adr-borrador (21); el estado sale de la línea «estado:» de cada archivo (grep). Una spec puede cubrir varios RF y los prefijos UX, DEV e INF cubren pantallas, soporte e infraestructura.

IA EN EL EQUIPO

13 agentes, cada uno con un rol y límites

Waggo

Especificación

requirements-analyst

Convierte un RF en spec con ejemplos y preguntas. Solo escribe en specs/.

Revisión

architecture-reviewer

Revisa contra ADR, capas y Definition of Done. Solo lectura.

Documentación

docs-keeper

Mantiene sincronizados el vault, las specs y los README.

Ciclo TDD · 6 agentes

tdd-orchestrator coordina · **backend-test-writer** y **mobile-test-writer** escriben pruebas rojas · **backend-implementer** y **mobile-implementer** hacen el verde y el refactor · **tdd-verifier** certifica cada fase.

Diseño

ui-ux-designer · design-implementer

Propuesta de pantallas y su llegada a la app, sin cambiar contratos.

Negocio

market-analyst

Estudio de mercado con fuente y fecha para cada cifra.

Comunicación

presentation-designer

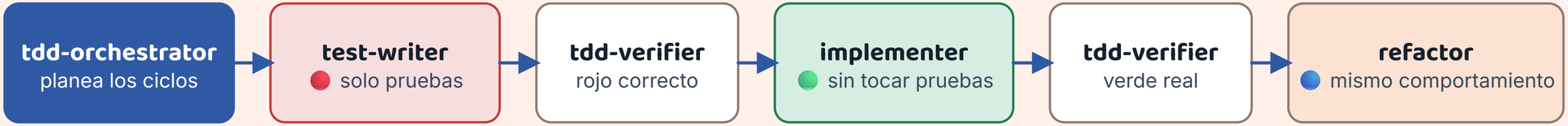
Presentaciones HTML con la marca; sin cifras inventadas.

Cómo se contó: archivos .md en waggo-workspace/.claude/agents (7) y .claude/agents/tdd (6) = 13. Los dos últimos (market-analyst y presentation-designer) aún no están commiteados en Git.

IA EN EL EQUIPO

Quien escribe la prueba no es quien la hace pasar

Waggo



.claude/hooks/tdd-guard.js (fragmento real)

```
if (mode === 'tests' && !isTestFile(path)) {
  console.error('TDD guard: los agentes de pruebas
    solo escriben pruebas. Bloqueado: ' + path);
  process.exit(2);
}
```

- El hook se engancha a **Edit y Write** de cada agente (PreToolUse)
- Un agente de pruebas no puede editar código de producción, y al revés
- Si una prueba parece mal, el implementador **lo informa**, no la cambia

Fuente: .claude/agents/tdd/*.md (frontmatter con hooks), .claude/hooks/tdd-guard.js (44 líneas), README del workspace. Flujo de commits: test → feat → refactor en la rama; el squash deja uno en main.

IA EN EL EQUIPO

Flujos reutilizables y reglas que los agentes leen

Waggo

7 skills (comandos)

`/new-spec`	crea la spec de un RF
`/implement-spec`	TDD, revisión, docs y título del PR
`/tdd`	ciclos para un cambio puntual
`/api-contract-change`	cambio coordinado del contrato
`/new-adr`	registra una decisión
`/sync-docs`	vault y README al día
`/workspace-status`	estado de los repos

Reglas que se leen antes de editar

- **2 checklists** en .claude/rules: backend y mobile
- **CLAUDE.md** y **AGENTS.md** de cada repo: comandos y reglas
- **Vault Waggo** (Obsidian): requisitos, arquitectura, ADR y glosario
- «Simplicidad primero»: el código más simple que se entienda de un vistazo
- Permisos compartidos: **git push denegado** y .env ilegibles

Cómo se contó: directorios dentro de .claude/skills (7) y archivos de .claude/rules (2). Permisos: .claude/settings.json (deny: git push, lectura de .env). El vault no estaba disponible al preparar la presentación.

IA EN EL EQUIPO

La IA propone y construye; el PO decide

Waggo

La IA hace

- Redacta specs con ejemplos, contrato y preguntas
- Escribe las pruebas rojas y el código mínimo
- Refactoriza y revisa contra los ADR y las capas
- Mantiene vault, specs y README sincronizados
- Propone diseño de pantallas (121 marcos SVG) y estudios

El PO decide

- Aprueba cada spec (Borrador → Lista) y resuelve preguntas
- **OpenStreetMap** como proveedor de mapas (2026-10-27)
- El **123** como línea de emergencia (2026-10-27)
- «Waggo» en el logo, una sola vez (2026-10-26)
- Qué se publica y cuándo: ningún push sin su aprobación

68 de 82 commits de los tres repos llevan un trailer de coautoría de Claude (31 de 36 en la API, 21 de 24 en la app, 16 de 22 en el workspace)

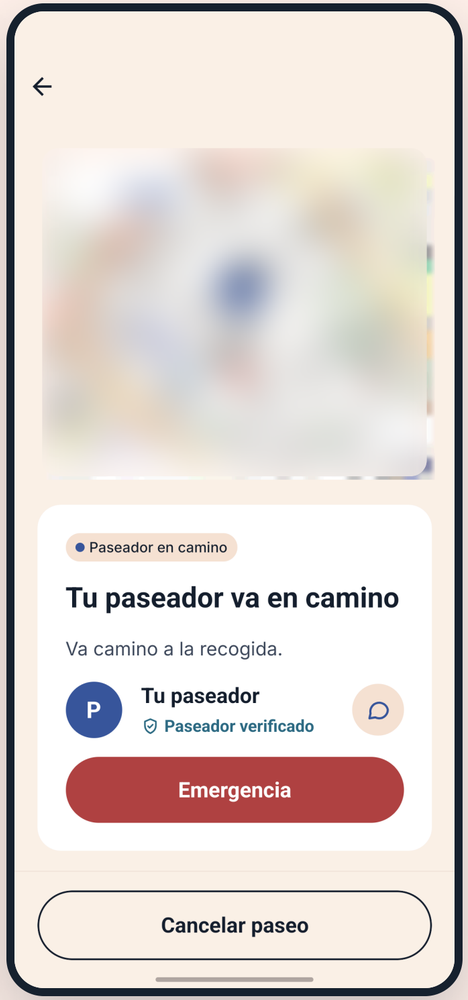
Cómo se contó: git rev-list --count -i --grep="co-authored-by: Claude" HEAD en cada repo (el del workspace, en su rama actual). Mide coautoría declarada, no cuántas líneas escribió la IA. Decisiones del PO: design/propuesta/00-resumen.md, 08-marca.md y specs/RF-012. 121 marcos SVG: inventario de design/propuesta/06-inventario-de-pantallas.md.

EVIDENCIA

Funciona en un celular real y en línea

Waggo





- **app.mywaggo.com** responde 200 (web de Expo en S3 y CloudFront)
- **api.mywaggo.com/health/ready** responde 200
- Capturas de la versión actual en un **Samsung Galaxy A35**
- Mapas difuminados: muestran la ubicación real de quien probó la app

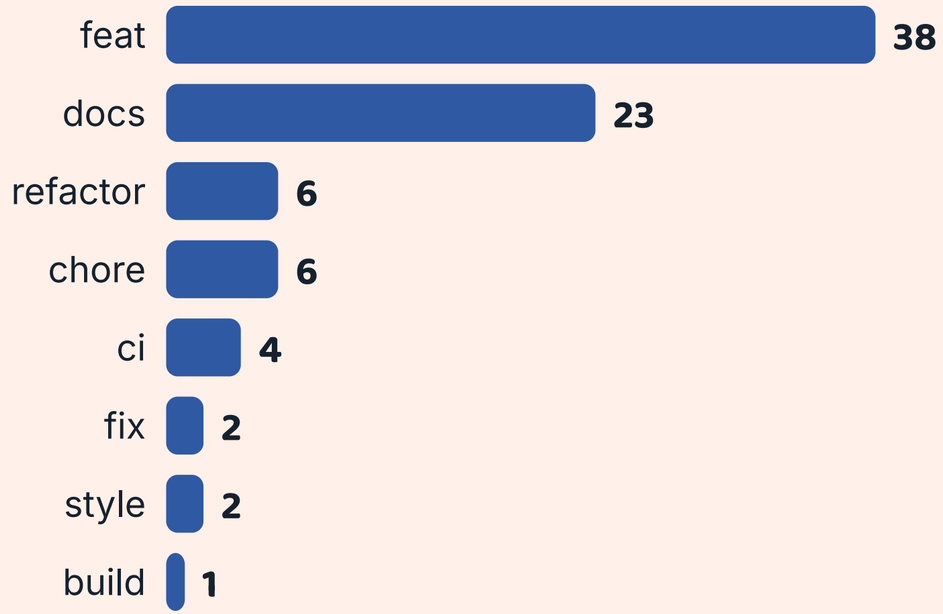
Fuente: presentaciones/_capturas (capturas del equipo, 2026-10-28); comprobación con curl el 2026-10-28.

EVIDENCIA

82 commits en 30 días, uno por PR

Waggo

Commits por tipo (Conventional Commits, tres repos)



36
waggo-api

24
waggo-mobile

22
workspace

- Primer commit: **2026-09-28**; último: 2026-10-28
- Cada PR entra como **un solo commit** (squash) con título Conventional Commits
- Commits como `feat(identity): sign in with Google via POST /auth/login/google (RF-001, RF-005)` enlazan con la spec

Cómo se contó: git rev-list --count HEAD por repo (waggo-workspace en su rama feature/agentes-mercado-presentaciones); tipo = prefijo del asunto del commit. Por el squash, un commit equivale a un PR integrado, no a una sesión de trabajo.

HONESTIDAD

Lo que todavía no está hecho

Waggo

TEMA	ESTADO HOY
Pagos reales	Pasarela **simulada**; el PO aún elige entre Stripe y MercadoPago
Verificación de paseadores	**Manual** por un administrador; sin biometría ni antecedentes automáticos
Notificaciones push	No existen: los avisos viven dentro de la app
Login con Google / Microsoft	API de Google en main; en la app aún no (spec en estado Lista)
Apps iOS y Android en tiendas	Fuera del alcance del despliegue actual; solo web en AWS
Tiles públicos de OSM	Solo para desarrollo y piloto; producción necesita proveedor propio
Vault y ADR de mapas	ADR de mapas en borrador; el vault no está en los repos

Fuente: specs/RF-015-RF-018, RF-002-RF-003, RF-012, RF-001-RF-005-login-social, INF-001 y specs/_adr-borrador-mapas-openstreetmap.md; git log de waggo-api (commit «sign in with Google», #32).

APÉNDICE

Cada cifra sale de un comando

Waggo

CIFRA	VALOR	CÓMO SE CONTÓ (2026-10-28)
Métodos de prueba de la API	626	`grep -rE "^\s*\[(Fact|Theory)" Tests (*.cs)`
Pruebas de la app	466	`grep -rE "^\s*(it|test)(\.each[^(]*)?\(" src (*.test.ts, *.test.tsx)`
Endpoints	44	`grep -rhoE "\.Map(Get|Post|Put|Patch|Delete)\(" Waggo.Api`
Specs	21	`ls specs/*.md sin _template, README ni _adr-borrador`
Agentes · skills	13 · 7	`ls .claude/agents .claude/agents/tdd .claude/skills`
Commits	82	`git rev-list --count HEAD (en cada repo)`
Coautoría de Claude	68	`git rev-list --count -i --grep="co-authored-by: Claude" HEAD`
Migraciones	19	`ls Migrations | grep -v "Designer|Snapshot"`

No se ejecutaron las suites ni se midió cobertura al preparar esta presentación: se cuentan pruebas definidas. Detalle completo en presentaciones/tecnica/guion.md.